

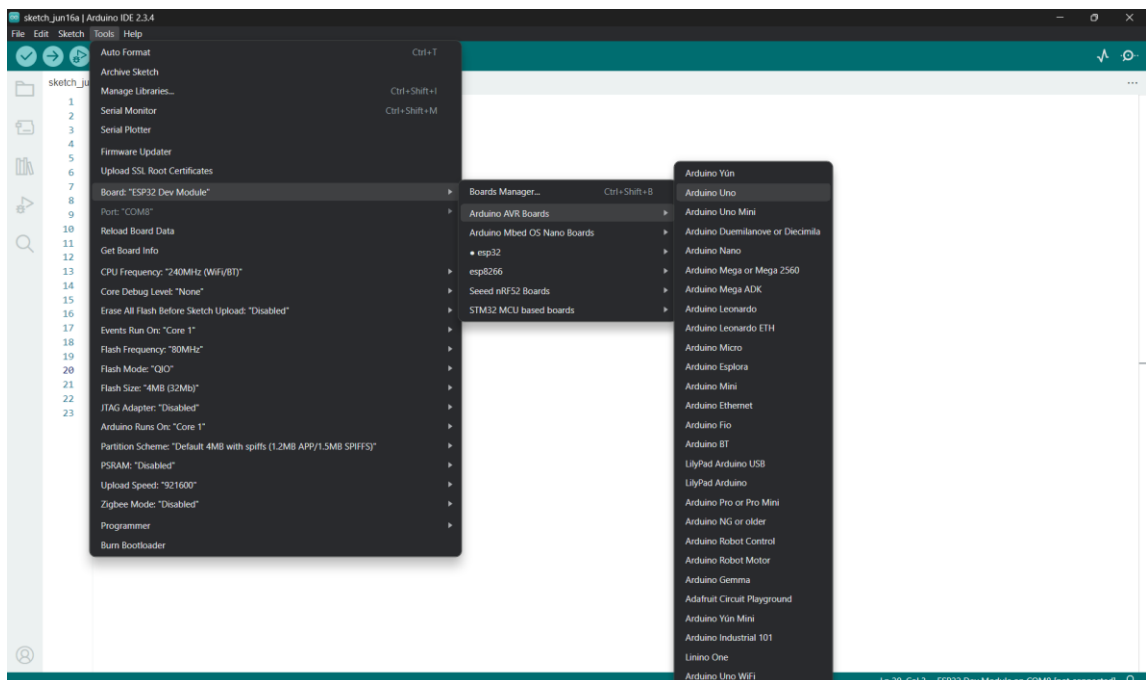
Catapult Coding Catalogue

Getting Started

- 1) Plug the microcontroller into the computer using its USB cable
- 2) Open Arduin IDE



- 3) Select "Arduino Uno" from **Tools -> Board -> Arduino AVR Boards -> Arduino Uno**



- 4) Select the correct COM Port in **Tools -> Ports**

HINT: Check the list of available COM Ports with the microcontroller UNPLUGGED from the computer. Then PLUG in the microcontroller and select the NEW COM Port which appears.

Preamble

- 1) Delete **all** of the existing code in the Arduino IDE
- 2) Copy and paste the following code:

```
#include <Servo.h>

#define servo_pin 9
#define button_pin 2

Servo servo;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);

  pinMode(button_pin, INPUT);
  servo.attach(servo_pin);

  servo.write(0);
  delay(500);
}

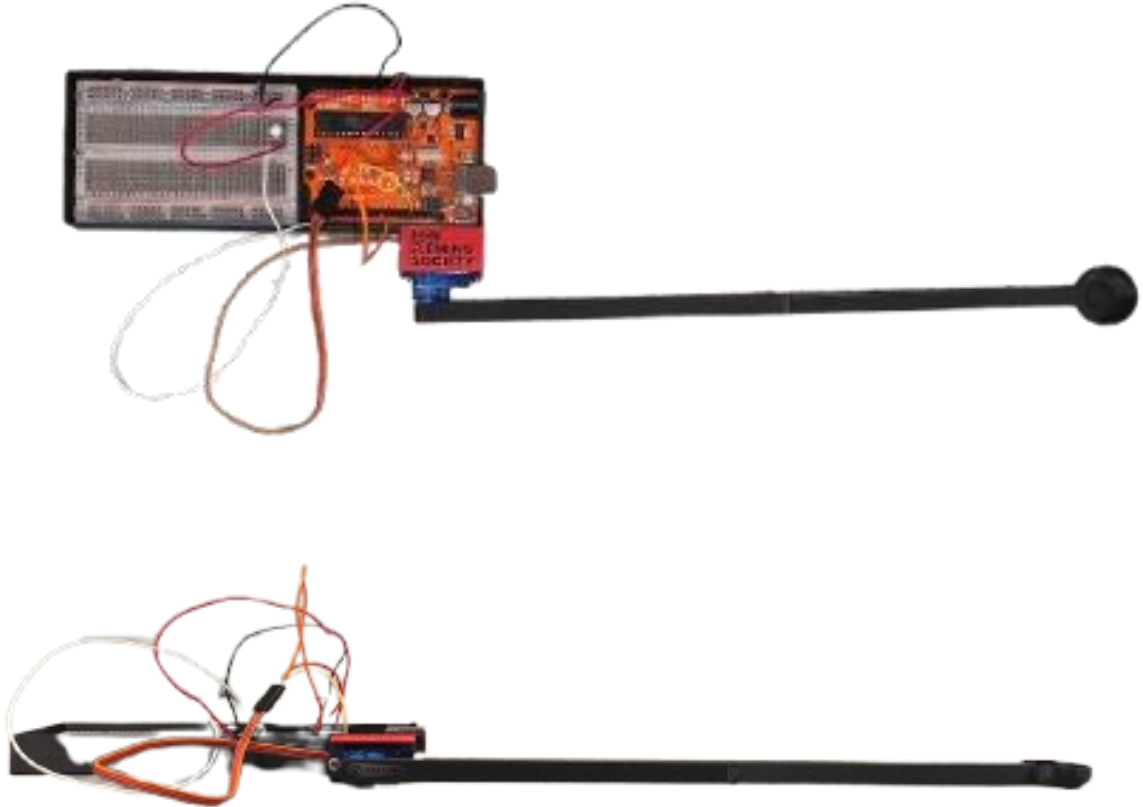
void loop() {
  // put your main code here, to run repeatedly:
}
```

Calibration

- 1) In the `void loop(){//Put your main code here: }` write the following code:

```
servo.write(0);
```

- 2) Attach the catapult arm as shown:



- 3) Ensure you put a **screw** through the servo horn securely!

Down to You!

Your objective is to make the catapult launch a projectile as far as possible when the button is pressed. To do this, you must:

- 1) Start the catapult at a known initial position
- 2) Wait and constantly check for a button press
- 3) IF the button is pressed, flip the catapult up to some end position
- 4) Then reset the catapult back to its initial position

To achieve this, you may find the following code blocks useful to put inside the

void loop(){//Put your main code here: }:

Moving the servo

- 1) Rotate the servo to x°

```
servo.write(x);
```

- 2) Rotate the servo on a specific trajectory: the servo will be rotated from x_1 to x_n sequentially, with there being a delay of t milliseconds between every rotation

```
int angle_trajectory = [x1, x2, x3, x4 ... xn];
int length = n;
for(int i = 0; i < length; i++){
    servo.write(angle_trajectory[i]);
    delay(t);
}
```

Detecting a button press

- 1) Detection method 1

```
button_state = digitalRead(button_pin);
if(button_state == LOW){
    // CODE IN HERE
    // will be executed if the condition is TRUE
}
```

- 2) Detection method 2

```
button_state = digitalRead(button_pin);
if(button_state == HIGH){
    // CODE IN HERE
    // will be executed if the condition is TRUE
}
```

Some other useful functions

- 1) Stops the program for x milliseconds

```
delay(x);
```

- 2) Prints the value of variable x to the Serial Monitor

```
Serial.println(x);
```